CORE
NEXUS

OPERATIONS
HANDBOOK

CORE Nexus – Installations- und Bereitstellungshandbuch

Windows · GitHub · Render Relay · LAN-Fallback

Dieses Handbuch beschreibt die Installation der CORE-Nexus-Server- und Agent-Programme sowie die Einrichtung des optionalen Render-Relays. Es bezieht sich auf das Paket CORE_Nexus_Logout_Locks_Dashboard.zip, das für diese Dokumentation untersucht wurde. Oberflächen und Dateinamen können sich in späteren Versionen ändern.

Version	Dokumentationsstand 15.10.2026
Geltungsbereich	CORE_Nexus_Logout_Locks_Dashboard.zip
Design	CORE Dark / Cyan

Inhalt

1. Überblick und Architektur
2. Voraussetzungen
3. Server auf dem Server-PC installieren
4. Agent auf einem Ziel-PC installieren
5. Render-Relay bereitstellen
6. Server und Agent mit Render verbinden
7. Testplan nach der Einrichtung
8. Updates ohne Datenverlust

1. Überblick und Architektur

CORE Nexus besteht aus drei Komponenten: dem Windows-Server mit Dashboard und lokaler Datenbank, dem Windows-Agenten auf jedem freizugebenden Ziel-PC sowie einem optionalen Relay-Web-Service auf Render. GitHub verwaltet den Relay-Quellcode; es vermittelt selbst keine Live-Fernsteuerung.

- Im selben lokalen Netzwerk versucht der Agent zuerst, den Server über die LAN-Erkennung zu finden. Die direkte Verbindung verwendet TCP-Port 47621 und UDP-Broadcast/Discovery auf Port 47622.
- Wenn die direkte LAN-Verbindung nicht gefunden oder erreicht wird, kann der Agent den Render-Relay verwenden, sofern beide Seiten korrekt konfiguriert sind.
- Render-Verbindungen verwenden WebSockets über TLS (`wss://`). Das schützt die Transportverbindung; die Anwendungsnachrichten sind damit nicht zusätzlich Ende-zu-Ende-verschlüsselt.
- Fernsteuerung erfordert lokale Zustimmung am Ziel-PC, Serverfreigabe und eine ausdrücklich geöffnete Fernsteuerungssitzung.

2. Voraussetzungen

- Windows-PC für den Server und Windows-PCs für die Agenten.
- Python 3 mit `py`-Launcher oder `python` im PATH. Während der Python-Installation muss Tcl/Tk mitinstalliert sein, damit Tkinter verfügbar ist.
- Internetverbindung während der Installation, damit die benötigten Python-Pakete installiert werden können.
- Für Geräte in unterschiedlichen Netzwerken: ein erreichbarer Render-Web-Service mit korrekt eingerichteten Umgebungsvariablen.
- Für LAN: Geräte müssen einander im lokalen Netzwerk erreichen können; WLAN-Client-Isolation oder Firewall-Regeln können Broadcasts und Verbindungen blockieren.

3. Server auf dem Server-PC installieren

1. Kopiere den Ordner `Server` aus dem Paket auf den gewünschten Windows-PC. Du kannst ihn beispielsweise unter `C:\CORE-Nexus\Server` ablegen.
2. Doppelklicke `install.bat`. Das Skript prüft Python und Tkinter, installiert die Server-Abhängigkeiten `pillow` und `websocket-client`, prüft die Syntax und erstellt die Installationsmarkierung.
3. Beim ersten Serverstart wird ein Owner-Konto angelegt. Wähle einen eindeutigen Benutzernamen und ein starkes Passwort. Das Passwort wird als Hash gespeichert.
4. Starte danach `start.bat`. Die GUI wird über `pyw`/`pythonw` gestartet, damit das Konsolenfenster geschlossen werden kann. Das Serverprogramm läuft weiter.
5. Bewahre den gesamten Ordner `Server/data/` auf. Dort liegt die lokale Datenbank mit Konten, Geräten, Freigaben und Protokollen.

4. Agent auf einem Ziel-PC installieren

1. Kopiere den Ordner `Agent` auf jeden Ziel-PC, den du verwalten möchtest.
2. Doppelklicke `install.bat`. Es prüft Python und Tkinter und installiert `pillow`, `mss`, `pynput` und `websocket-client`. Die Fernsteuerungsbibliotheken müssen erfolgreich installiert sein.
3. Starte `start.bat`. Das Agent-Fenster zeigt den Verbindungsstatus, die Geräte-ID und den verwendeten Verbindungsweg.
4. Beim ersten Start erscheint eine lokale Zustimmungsabfrage. Der lokale Benutzer muss ausdrücklich zustimmen. Eine Ablehnung hält den Agenten geschlossen.
5. Gib das Gerät danach zusätzlich im Server-Dashboard frei. Lokale Zustimmung und Serverfreigabe sind getrennte Voraussetzungen.

5. Render-Relay bereitstellen

1. Lade den Inhalt des Ordners `GitHub` aus dem Paket in das Stammverzeichnis deines privaten GitHub-Repositories. `render.yaml`, `requirements.txt` und der Ordner `relay/` müssen im Repository-Stamm liegen.
2. Öffne Render und wähle `New +` → `Blueprint`. Verbinde GitHub, wähle das Repository und bestätige die aus `render.yaml` erkannten Einstellungen.
3. Die YAML-Datei definiert einen Python-Web-Service mit `pip install -r requirements.txt`, Startbefehl `uvicorn relay.main:app --host 0.0.0.0 --port $PORT` und Health-Check `/health`. Wenn du bereits einen Dienst hast, prüfe die Einstellungen, bevor du etwas neu erstellst.
4. Öffne im Render-Dienst den Bereich `Environment`. Stelle sicher, dass `CORE_NEXUS_RELAY_TOKEN` und `CORE_NEXUS_DASHBOARD_TOKEN` vorhanden sind. Sie müssen unterschiedlich und geheim sein. Wenn sie fehlen, setze lange, zufällige Werte.
5. Nach dem Deploy teste `https://DEIN-DIENST.onrender.com/health`. Erwartet wird eine JSON-Antwort mit `status: ok`. Das beweist nur, dass der Dienst erreichbar ist; es beweist noch nicht, dass ein Agent verbunden ist.
6. Öffne `https://DEIN-DIENST.onrender.com/` für das Relay-Dashboard. Gib dort den Wert von `CORE_NEXUS_DASHBOARD_TOKEN` ein.

6. Server und Agent mit Render verbinden

In beiden lokalen Ordnern gibt es eine Datei `relay_config.json`. Die geprüfte Paketversion enthält dort einen Beispiel-Endpunkt und den Platzhalter `PASTE_RENDER_TOKEN_HERE`. Dieser Platzhalter muss ersetzt werden, sonst ist die Relay-Konfiguration ungültig.

1. Öffne `Server/relay_config.json` und `Agent/relay_config.json` mit einem Texteditor.
2. Setze `relay_url` auf deine tatsächliche Render-Adresse mit WebSocket-Pfad, zum Beispiel `wss://DEIN-DIENST.onrender.com/ws`. Verwende nicht die Beispieladresse, wenn sie nicht dein eigener Dienst ist.
3. Setze `relay_token` auf den exakten Wert von `CORE_NEXUS_RELAY_TOKEN` aus Render. Die beiden lokalen Dateien müssen denselben Wert verwenden.
4. Setze `room_id` in beiden Dateien exakt gleich. Verwende eine lange, zufällige Kennung und veröffentliche sie nicht unnötig.
5. Belasse `enabled` auf `true`, wenn Render verwendet werden soll. Speichere die Dateien und starte Server und Agent neu.

WICHTIG: Die echten lokalen `relay_config.json`-Dateien enthalten ein Geheimnis. Lade sie niemals in ein öffentliches oder privates GitHub-Repository hoch, wenn das Repository nicht ausdrücklich als Geheimnisspeicher abgesichert ist. Bewahre sie lokal auf und verwende für GitHub nur Beispielkonfigurationen ohne echte Tokens.

7. Testplan nach der Einrichtung

1. Starte den Server und melde dich am Dashboard an.
2. Starte einen Agenten im selben LAN. Der Agent sollte `DIRECT LAN` oder einen LAN-Verbindungsweg anzeigen. Dieser Verkehr benötigt Render nicht.
3. Bestätige lokale Zustimmung und gib das Gerät im Dashboard frei.
4. Öffne ausdrücklich eine Fernsteuerungssitzung und prüfe Bild, Maus und Tastatur.
5. Teste danach einen Agenten in einem anderen Netzwerk, beispielsweise über einen separaten Hotspot. Der Agent sollte bei fehlender LAN-Verbindung auf Render wechseln.
6. Öffne das Render-Dashboard und kontrolliere, ob die externe Verbindung gezählt wird. LAN-Geräte werden dort absichtlich nicht gezählt.

8. Updates ohne Datenverlust

- Beende Server oder Agent vor dem Austausch von Dateien.
- Server-Updates: Sichere `Server/data/` vor dem Ersetzen. Überschreibe den Datenordner nicht mit einem leeren Ordner aus einem neuen Paket.
- Agent-Updates: Bewahre `device_id.json`, `consent.json` und die echte `relay_config.json` auf, sofern du die Geräteidentität, Zustimmung und Relay-Einstellungen beibehalten möchtest.
- Nach Updates `start.bat` erneut ausführen. Wenn der Start ohne Konsole erfolgt und das Programm nicht erscheint, starte die Python-Datei vorübergehend aus einer geöffneten Eingabeaufforderung, um die Fehlermeldung zu sehen.